GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unidad 02 — Gestión de procesos
Bloque 02

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea

Comunicación con procesos

La idea en una frase

> Un proceso hijo puede **leer de su stdin** y **escribir en su stdout**; con `communicate()` le pasas datos por un tubo y lees su respuesta por el otro.

Los procesos no comparten memoria, pero el sistema operativo les presta **pipes** (tuberías): el stdin, stdout y stderr de cada proceso pueden conectarse a Python. Así un proceso "pregunta" y el otro "responde".

Conectar los tubos

stdin=PIPE, stdout=PIPE

tubos.py — Python

```
import subprocess
proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)
```

1. `stdin=PIPE` : Python podrá **escribir** en el stdin del hijo
2. `stdout=PIPE` : Python podrá **leer** el stdout del hijo
3. `text=True` : los datos viajan como **texto**, no bytes

ℹ️ El hijo ejecuta `print(input().upper())` : lee una línea, la pasa a mayúsculas y la escribe en su stdout.

Enviar y recibir con communicate()

comunica.py — Python

```
import subprocess
proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True)
salida, error = proceso.communicate(
    input="hola mundo")
print(f"Respondió: {salida.strip()}")
# → "HOLA MUNDO"
print(f"Error: {error}")
# → None (todo bien)
```

1. `communicate(input=...)` **escribe** en el stdin del hijo y cierra el tubo de entrada
2. **Espera** a que el hijo termine
3. Devuelve la tupla **(stdout, stderr)**

⚠️ Después de communicate() el proceso ya terminó: **no puedes volver a comunicarte** con él.

El flujo de datos y los deadlocks

Flujo de datos

flujo — Pipes

```
Python ──stdin──► hijo
       ("hola mundo")
hijo ──stdout──► Python
       ("HOLA MUNDO")
```

⚠️ Cuidado con los deadlocks

Si el hijo escribe mucho en stdout y **nadie lo lee**, el pipe se llena y el hijo se bloquea esperando a que alguien lo vacíe.

✅ `communicate()` lee todo y te evita ese problema. Por eso con run() se usa `capture_output=True` y con Popen, `communicate()`.

¿Cuándo usar cada herramienta?

Situación	Herramienta
Necesitas el resultado de un comando y puedes esperar	`subprocess.run(..., capture_output=True)`
El proceso vive en segundo plano y no necesitas sus datos	`subprocess.Popen(...)` + `wait()`
Necesitas enviarle datos y/o leer su respuesta	`Popen(..., stdin=PIPE, stdout=PIPE)` + `communicate()`

🧠 **Mini-chequeo:** ¿qué significa stdin=subprocess.PIPE y stdout=subprocess.PIPE? ¿Qué devuelve communicate(input="texto")?

> Que Python podrá **escribir** en el stdin del hijo y **leer** su stdout · communicate() devuelve la tupla **(stdout, stderr)**.

El muro del aislamiento

La burbuja de memoria, en acción

aislado.py — Python

```python
import multiprocessing
puntuacion = 10  # variable global
def sumar_puntos():
    global puntuacion
    puntuacion += 5
    print(f"Hijo: La puntuación es {puntuacion}")
if __name__ == "__main__":
    p1 = multiprocessing.Process(target=sumar_puntos)
    p1.start()
    p1.join()
    print(f"Padre: La puntuación es {puntuacion}")
```

```
$ python aislado.py
Hijo: La puntuación es 15
Padre: La puntuación es 10
```

1. Al hacer `start()`, el SO hace un **clon** de toda la memoria en ese instante
2. El hijo suma 5 sobre **su clon** (15); al morir, su burbuja de memoria se destruye
3. El padre imprime **su variable original**, que nunca fue modificada

⚠️ Los procesos **no comparten variables globales**: el SO clona la RAM.

Rompiendo el muro: Manager

Comunicación entre procesos (IPC)

servidor.py — Python

```
import multiprocessing
def registrar_usuario(dic, nombre, edad):
    # Esto SÍ modifica la memoria que ven todos
    dic[nombre] = edad
    print(f"Hijo registró a {nombre}")
if __name__ == "__main__":
    with multiprocessing.Manager() as manager:
        # diccionario proxy que cruza la burbuja
        buzones = manager.dict()
        p1 = multiprocessing.Process(
            target=registrar_usuario,
            args=(buzones, "Alice", 25))
        p2 = multiprocessing.Process(
            target=registrar_usuario,
            args=(buzones, "Bob", 30))
        p1.start(); p2.start()
        p1.join(); p2.join()
        print(f"Diccionario final: {buzones}")
```

1. `multiprocessing.Manager()` : crea un **proceso servidor invisible** que gestiona la memoria compartida
2. `manager.dict()` : un diccionario **proxy** que cruza la burbuja de memoria
3. Los hijos lo reciben como **argumento normal**: `args=(buzones, "Alice", 25)`

✅ Al terminar, el padre **SÍ ve los cambios:**
`{'Alice': 25, 'Bob': 30}`

Condición de carrera y Lock

Solo un proceso en la sección crítica

contador.py — Python

```python
import multiprocessing, time
def incrementar(cont, candado):
    for _ in range(100):
        with candado:  # acquire() y release() automáticos
            valor = cont.value    # sección crítica:
            time.sleep(0.001)     # leer → calcular → escribir
            cont.value = valor + 1
if __name__ == "__main__":
    cont = multiprocessing.Value('i', 0)
    candado = multiprocessing.Lock()
    procesos = []
    for i in range(5):
        p = multiprocessing.Process(
            target=incrementar, args=(cont, candado))
        procesos.append(p); p.start()
    for p in procesos:
        p.join()
    print(f"Valor final: {cont.value}") # siempre 500
```

⚠️ Una **condición de carrera** (race condition) ocurre cuando varios procesos modifican un dato a la vez: se machacan y el resultado es impredecible (143, 290...). Sin el `Lock`, el contador final no sería 500.

1. El proceso **pide el candado**: `acquire()`; si otro lo tiene, se queda bloqueado esperando
2. Modifica la variable compartida: es la **sección crítica**
3. Suelta el candado: `release()`; con `with lock:`, acquire y release se hacen **automáticamente**

Mini-chequeo: multiprocessing

🧠 Las cuatro del muro:

1. ¿Por qué usar `if __name__ == "__main__"` al crear procesos en Windows?
2. Si un proceso hijo modifica una lista global estándar, ¿el padre verá la lista modificada?
3. ¿Qué hace `manager.dict()`?
4. ¿Para qué sirve un `Lock` en programación concurrente?

🔄 **Respuestas:**

- En Windows no hay **fork**: sin el bloque, cada hijo reimporta el script completo y vuelve a crear procesos (bucle infinito)
- No: por el **aislamiento**, el hijo modifica su propio clon en su burbuja de RAM
- Crea un diccionario **proxy** del SO para compartir datos entre procesos de forma segura
- Evitar **condiciones de carrera**: solo un proceso a la vez entra en la **sección crítica**

Ejemplo resuelto: estado compartido con procesos

Un almacén que cuatro procesos actualizan sin perder una escritura

ℹ️ `manager.dict()` + cuatro `Process` + `Lock` + JSON de persistencia: el patrón que reutilizarás en el servidor con procesos de la **UD 6**.

1. **El trabajador**: cada proceso hijo recibe el almacén, el candado y su lista de pedidos
2. **Guardar y cargar**: volcar el almacén a JSON y recuperarlo al arrancar
3. **El main**: crea `Manager` y `Lock`, lanza los 4 procesos, espera con `join()` y guarda

Paso 1: el trabajador

La sección crítica, dentro de un solo candado

almacen.py (trabajador) — Python

```
import multiprocessing, random, time
def trabajador(almacen, candado, pedidos):
    for fruta, unidades in pedidos:
        time.sleep(random.uniform(0.01, 0.05))
        with candado:              # sección crítica
            total = almacen.get(fruta, 0)
            almacen[fruta] = total + unidades
        print(f"  [{multiprocessing.current_process().name}] "
              f"+{unidades} {fruta}")
```

1. `with candado:` protege la **lectura y la escritura juntas**. Si leyeras fuera, dos procesos leerían el mismo valor previo y el último machacaría al primero
2. `current_process().name` te dice qué proceso habla ("trabajador-1", "trabajador-2"...)
3. El trabajador no sabe que hay otros: solo usa `get` y asignación sobre el proxy

⚠️ Si quitas el `with candado:`, el total cambia en cada ejecución.

Paso 2: persistencia

El almacén vive en memoria: si apagas, desaparece

almacen.py (persistencia) Python

```python
def guardar_almacen(almacen, fichero="almacen.json"):
    with open(fichero, "w", encoding="utf-8") as f:
        json.dump(dict(almacen), f, ensure_ascii=False, indent=2)
    print(f"Almacén guardado en {fichero}")
def cargar_almacen(fichero="almacen.json"):
    try:
        with open(fichero, encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        return {}
```

⚠️ `json.dump(almacen)` falla: un **proxy** de `Manager` no es un dict. `dict(almacen)` hace la "foto" del estado actual y devuelve un dict normal y corriente.

Paso 3: el main que lo pone todo en marcha

almacen.py (main) Python

```
if __name__ == '__main__':
    cargado = cargar_almacen()
    with multiprocessing.Manager() as manager:
        almacen = manager.dict(cargado)   # arranca del turno anterior
        candado = multiprocessing.Lock()
        reparto = [
            [("manzanas", 3), ("uvas", 2)],
            [("peras", 5)],
            [("manzanas", 1), ("naranjas", 4)],
            [("uvas", 1), ("peras", 2), ("naranjas", 2)],
        ]
        procesos = []
        for i, pedidos in enumerate(reparto, start=1):
            p = multiprocessing.Process(
                target=trabajador,
                args=(almacen, candado, pedidos),
                name=f"trabajador-{i}")
            procesos.append(p)
            p.start()
        for p in procesos:                # imprescindible antes de leer
            p.join()
        print(f"Total del turno: {dict(almacen)}")
        guardar_almacen(almacen)
```

La traza y la salida

El orden varía; el total, no

```
$ python almacen.py
Estado previo: {}
[trabajador-1] +3 manzanas
[trabajador-3] +1 manzanas
[trabajador-2] +5 peras
[trabajador-4] +1 uvas
Total del turno: {'manzanas': 4, 'uvas':
3,
'peras': 7, 'naranjas': 6}
Almacén guardado en almacen.json
```

1. `start()` × 4: el SO crea 4 burbujas, pero el proxy las conecta con **un único** almacén
2. Cada `get` y cada escritura viajan por mensaje al **proceso del Manager**
3. `join()` × 4 antes de leer: sin él, el padre leería con escrituras en vuelo

✅ El orden de las líneas cambia en cada ejecución; el **total final es siempre el mismo**: esa es la garantía del candado.

Quita el candado y mira la catástrofe

⚠️ Sin el `with candado:`, dos procesos leen el mismo valor previo y el segundo machaca el primero: **manzanas: 2** en vez de 4.

ℹ️ Ejecuta el programa tres o cuatro veces: **nunca** sabrás qué escritura se pierde. Es la condición de carrera de la UD 4, ahora entre **burbujas de memoria**.

🧠 Mini-chequeo del ejemplo:

- **¿Almacén por argumento?** Las globales se clonan; el proxy cruza la burbuja
- **¿join() antes de leer?** Para no dejar escrituras en vuelo
- **¿El segundo día arranca con datos?** Porque `cargar_almacen()` alimenta el `manager.dict()`

Compatibilidad Windows / Linux

La idea en una frase

> Los ejemplos están escritos para **Windows**, pero los conceptos (subprocess.run, Popen, PID, communicate) son **idénticos en todos los sistemas**: solo cambia el comando.

ℹ️ `notepad.exe` no existe en Linux y `ls` no existe en Windows. Aprende a **sustituir el comando** y el código funciona igual.

La tabla de comandos equivalentes

Windows	Linux / macOS
`notepad.exe`	gedit, nano, xed
`calc.exe`	gnome-calculator, bc
`ping -n 5 8.8.8.8`	`ping -c 5 8.8.8.8`
`ipconfig`	`ip addr`, `ifconfig`
`where python`	`which python`
`cmd /c mkdir`	`mkdir` (es ejecutable real)
`start http://...`	`xdg-open http://...`
`dir`	`ls`

✅ **Sustituye el comando correspondiente y el código funciona igual.**

El truco del shell: mkdir en Windows

En **Windows**, `mkdir` es un **comando interno de cmd**: no hay un mkdir.exe, hay que invocarlo con `cmd /c mkdir ...`

En **Linux**, `mkdir` es un **ejecutable real** (/usr/bin/mkdir): se lanza directamente

⚠️ **shell=True con cuidado:** solo para comandos internos del shell (start, dir). Con texto del usuario, un atacante podría **inyectar comandos**. Pasa siempre la lista de argumentos.

shell.py — Python

```python
import subprocess
# Windows: cmd /c ejecuta comandos internos
r = subprocess.run(
    ["cmd", "/c", "mkdir", "prueba_psp"],
    capture_output=True, text=True)
print(f"Creada con código: {r.returncode}")
# En Linux: ["mkdir", "prueba_psp"] directo
```

Lo que nunca cambia

Concepto	Windows	Linux
`subprocess.run([...])`	✅	✅
`subprocess.Popen([...])`	✅	✅
`capture_output=True`, `text=True`	✅	✅
`timeout`, `returncode`	✅	✅
`communicate(input=...)`	✅	✅
`wait()`, `poll()`, `terminate()`, `kill()`	✅	✅
`proceso.pid`	✅	✅

✅ Aprende los conceptos con notepad.exe y calc.exe y, cuando toques Linux, solo cambia la lista de comandos.

Sé el código: dos aplicaciones a la vez

Traza cada paso antes de ejecutar

doble.py — Python

```
import subprocess, time
print("🚀 Abriendo bloc de notas...")
notepad = subprocess.Popen(["notepad.exe"])
print(f"  → PID: {notepad.pid}")
print("🚀 Abriendo calculadora...")
calc = subprocess.Popen(["calc.exe"])
print(f"  → PID: {calc.pid}")
for i in range(5):
    print(f" Python haciendo cosas... ({i+1}/5)")
    time.sleep(1)
notepad.terminate()
calc.kill()
print("Hecho 🏁")
```

La traza, paso a paso

Tres procesos independientes conviviendo

1. `Popen(["notepad.exe"])`: el SO busca el ejecutable, lo carga en memoria, asigna **PID 12345** y Python recibe el control al instante
2. `Popen(["calc.exe"])`: lo mismo, **PID 12346**
3. Python hace 5 cosas mientras ambos están abiertos: **3 procesos independientes** (Python, notepad, calc)
4. `terminate()` cierra el bloc · `kill()` mata la calculadora

✅ **Python no se bloquea.** Mientras los dos programas están abiertos, Python sigue imprimiendo: cada uno con su PID, su memoria y su turno de CPU.

🥊 run() vs Popen(): el ring

ℹ️ **run():** "¡Soy el rey de la simplicidad! Lanzas, esperas y ¡zas! tienes el resultado."

ℹ️ **Popen():** "Mientras tú esperas, yo lanzo y sigo. ¿3 programas a la vez? Los lanzo, me voy a café y luego los mato."

⚠️ **Moraleja:** run() para comandos rápidos que necesitan respuesta. Popen() para procesos que deben vivir en segundo plano.

Mini-chequeo: práctica

🧠 Las tres del ring:

1. En el "Sé el código", ¿cuántos procesos hay vivos mientras Python imprime sus 5 mensajes?
2. ¿Cuándo usarías run() y cuándo Popen()?
3. ¿Qué devuelve `os.getppid()`?

🔁 **Respuestas:**

- **3**: Python, el bloc de notas y la calculadora
- run() para respuestas inmediatas; Popen() para segundo plano
- El **PID del proceso padre** (el que lanzó tu programa)

Sé el proceso: eres el PID 12345

Nace, vive y muere un proceso

ℹ️ 🎯 *Eres un proceso Python recién lanzado con* `subprocess.Popen(["python", "calcula.py"])` *y acaban de asignarte el PID 12345. ¿Qué te pasa?*

1. El SO te crea: PID **12345** y burbuja de memoria reservada
2. **NUEVO** → **LISTO**: esperas turno en la cola del planificador
3. La CPU te toca: **EJECUCIÓN**, ejecutas calcula.py
4. Llamas a `input()`: te bloqueas (**BLOQUEADO**) esperando al usuario
5. El usuario pulsa ENTER: la E/S termina y vuelves a **LISTO**
6. Última ronda de CPU: **EJECUCIÓN** → **TERMINADO**
7. Tu padre recoge el retorno con `proceso.wait()`: código **0**

ℹ️ 💡 **Ahora tú:** ¿y si tu padre lanza a la vez otros dos procesos (calc.exe y notepad.exe)? Los tres avanzáis turnándose la CPU (**concurrencia**) y, si hay varios núcleos, alguno ejecutará de verdad a la vez (**paralelismo**).

🔥 Fireside chat: Proceso vs Hilo

Quién es más ligero

ℹ️ **Proceso:** — Yo soy la unidad completa: memoria propia, recursos, PID. Si me cuelgo, tú ni te enteras.

✅ **Hilo:** — Cierto, pero yo comparto variables directamente. Tú necesitas pipes y communicate() para hablar con tu vecino.

✅ **Hilo:** — ¿Y para qué tanta burbuja? Yo vivo **dentro** de un proceso y comparto su memoria. Soy mucho más barato de crear.

ℹ️ **Proceso:** — Y mi paralelismo con multiprocessing es real: los hilos comparten el **GIL**.

ℹ️ **Proceso:** — Mis fallos no tumban a nadie. Tú, si se te va un puntero, te llevas por delante al proceso entero.

⚠️ **Moraleja:** el proceso **aisla y paraleliza de verdad**; el hilo es **ligero y comparte memoria**. Los hilos los veremos más adelante en el temario.

🕵️ ¿Quién soy?

🕵️ Endevina qué concepto de la unidad soy:

1. Soy el identificador único que el SO asigna a cada proceso
2. Soy el estado en el que el proceso espera un recurso (I/O, socket, sleep)
3. Soy el mecanismo por el que una sola CPU parece ejecutar varios procesos a la vez
4. Soy el módulo de Python que reparte tareas en procesos paralelos reales
5. Soy el tubo que conecta la salida de un proceso con la entrada de otro
6. Soy la herramienta de sincronización que evita que dos procesos modifiquen la memoria compartida al mismo tiempo (condición de carrera)
7. Soy el proceso que ya terminó pero que su padre no ha recogido todavía

🔄 **Respuestas:** 1) **PID** · 2) **BLOQUEADO** · 3) El **planificador** con la concurrencia (time slices) · 4) **multiprocessing** · 5) El **pipe** · 6) El **Lock** (multiprocessing.Lock) · 7) El **zombie**

CONRAD vs el mundo: "maté el proceso y no volvió"

⚠️ *"He lanzado mi programa con subprocess.run y no me devuelve el control."* ¡Pues claro! **run() espera** a que el comando termine: si lanzas notepad.exe, no se cierra nunca. Si querías lanzar y seguir, eso es **Popen**, no run().

⚠️ *"Puse shell=True para abrir el navegador y ahora un comando borra mis archivos."* shell=True delega en el intérprete: con datos del usuario es un **agujero de seguridad**. Lista de argumentos, nunca strings concatenados. O `cmd /c`, que para eso está.

😈 CONRAD vs el mundo (II): "el padre no ve mi cambio"

⚠️ *"Modifico una variable global en mi proceso hijo y en el padre vale 0."* ¡Bienvenido al **aislamiento de memoria**! Los procesos se ejecutan en burbujas separadas: el hijo clona la variable, la modifica en su burbuja y, al morir, se pierde. Si no usas un **Manager** (IPC), tus cambios mueren en la burbuja del hijo.

```
$ python aislado.py
Hijo: La puntuación es 15
Padre: La puntuación es 10
```

⚠️ *"¿Será que mi proceso se ha convertido en zombie?"* Míralo con `proceso.poll()`: si devuelve el código de retorno, ya terminó y solo falta recogerlo con wait() o poll(). **Zombie sin recoger = entrada ocupada** en la tabla de procesos. A diagnosticar.

🧠 Atrévete a pensar

1. ¿Por qué un proceso no puede compartir memoria directamente con otro?
2. ¿Qué pasa si un hijo muere y su padre nunca llama a wait() ni poll()?
3. ¿Cuándo usarías multiprocessing en lugar de subprocess?
4. ¿Por qué shell=True es peligroso con datos del usuario?
5. ¿Qué ventaja tiene Popen sobre run() para lanzar 3 aplicaciones a la vez?

💡 **Soluciones**

- **1)** Burbuja aislada: se comunican con mecanismos externos (pipes, sockets, archivos)
- **2)** Queda **zombie**: su entrada sigue en la tabla de procesos
- **3)** multiprocessing para **paralelismo real**; subprocess para **lanzar programas**
- **4)** Un dato con `;`, `&` o `|` ejecuta comandos extra (inyección)
- **5)** Popen devuelve el control al instante

🧠 Atrévete a pensar: multiprocessing

1. ¿Qué ocurre si ejecutas `multiprocessing.Process` en Windows sin incluir el bloque `if __name__ == "__main__"`?
2. ¿Por qué un `manager.dict()` necesita un `Lock` si ya es un objeto especial de memoria compartida?
3. ¿Qué diferencia existe entre el método `wait()` de `Popen` y el método `join()` de `Process`?

💡 **Soluciones**

- **1)** En Windows no hay fork: el hijo reimporta el script completo y vuelve a crear procesos → **bucle infinito** hasta agotar la RAM
- **2)** El Manager permite el acceso concurrente, pero no evita condiciones de carrera en operaciones compuestas (leer → calcular → escribir): el **Lock** garantiza la exclusión mutua
- **3)** Conceptualmente lo mismo (bloquear al padre hasta que acabe el hijo), pero `wait()` es de **subprocess** (procesos del SO) y `join()` de **multiprocessing** (nativos de Python)

🤷 No hay preguntas tontas

ℹ️ ❓ **¿Cuántos procesos puede tener mi sistema?**
Depende de la **memoria RAM**: cada proceso ocupa memoria. En Windows, el Administrador de tareas los muestra todos.

ℹ️ ❓ **¿Y si el padre muere antes que los hijos?**
Son **huérfanos**. En Windows los gestiona el sistema; en Linux los adopta **init** (PID 1) o systemd, que recoge sus estados al terminar.

ℹ️ ❓ **¿Puedo lanzar cualquier programa?**
Sí, cualquier ejecutable. Pero debe estar en el **PATH** o dar la ruta completa.

ℹ️ ❓ **Si multiprocessing duplica la memoria al crear procesos, ¿es muy costoso?**
Sí: crear procesos es más pesado que crear hilos (**overhead** de RAM y CPU). Por eso en producción se usan **pools de procesos** (ProcessPoolExecutor) que los reutilizan.

Preguntas de entrevista de trabajo

💼 Preguntas reales de programador Python júnior:

1. ¿Qué es un proceso y en qué se diferencia de un programa?
2. Explica el ciclo de vida de un proceso
3. ¿Qué diferencia hay entre computación paralela y distribuida?
4. ¿Cómo lanzarías un programa externo desde Python y leerías su salida?
5. ¿Qué diferencia hay entre run() y Popen()? ¿Cuándo usarías cada uno?
6. ¿Cómo comunicarías dos procesos entre sí?

ℹ️ **Las "preguntas reina" son la 4 y la 5:** run([...], capture_output=True, text=True) → resultado.stdout · y la moraleja: run() para respuestas inmediatas, Popen() para segundo plano.

Entrevista: multiprocessing

Las tres de memoria compartida:

1. "¿Por qué en Python el multiproceso permite **paralelismo real** mientras que los hilos sufren por el GIL?"
2. "¿Cómo comunicarías dos procesos de Python que necesitan **compartir un diccionario** de datos en tiempo real?"
3. "¿Qué es una **condición de carrera** (race condition) y cómo se previene en un entorno multiproceso?"

Cómo encararlas:

- Para la 1: cada proceso levanta su **propio intérprete** de Python (GIL independiente por núcleo)
- Para la 2: `multiprocessing.Manager()` y sus objetos **proxy** (`manager.dict()`)
- Para la 3: define la **sección crítica** y explica cómo `multiprocessing.Lock()` garantiza la exclusión mutua

Resumen — Bloque 02 completado

Lo que hemos aprendido

🔌 Comunicación

stdin/stdout con PIPE · communicate(input=...) · deadlocks

🪟 🐧 Windows / Linux

Misma API, otro comando · cmd /c y shell=True con cuidado

🥊 Práctica

Traza de procesos conviviendo · run() vs Popen() · zombie

🧬 multiprocessing (IPC)

Manager().dict() · Lock · ejemplo resuelto del almacén con persistencia JSON

💼 Cierre

Proceso vs hilo · ¿quién soy? · preguntas de entrevista

✅ **Ya sabes lanzar, vigilar, alimentar y matar procesos, y compartir memoria entre ellos.**
Ahora: boletines de práctica. 🚀

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //